Investigating Multithreaded PostgreSQL

Concepts | Architectures | WIP

Thomas Munro
Free range open source database hacker at Microsoft
pgconf.dev 2025 Montréal

Concepts

A long time ago in a galaxy far, far away…

> A last aspect of our design concerns the operating system process structure. Currently, POSTGRES runs as one process for each active user. This was done as an expedient to get a system operational as quickly as possible. We plan on converting POSTGRES to use lightweight processes available in the operating systems we are using. These include PRESTO for the Sequent Symmetry and threads in Version 4 of Sun/OS.
>
> - The Implementation of POSTGRES, Stonebraker et al, covering 1985-1990

- Some Unixen had developed their own incompatible threading APIs, some had none, so portability was thorny
- POSIX standardised <pthread.h> in 1995
- Windows' threading API appeared in 1993
- C standardised <threads.h> in 2011, but it is still missing from at least one important system

1 2 3

	Start a process by forking/cloning the current process	Start a different executable in a child process	Start a thread in the current process
POSIX	fork()	fork() + exec() vfork() + exec() posix_spawn()	pthread_create()
Win32		CreateProcess()	CreateThread()

A fork() in the road

Andrew Baumann, Microsoft Research
Jonathan Appavoo, Boston University
Orran Krieger, Boston University
Timothy Roscoe, ETH Zurich

ABSTRACT

The received wisdom suggests that Unix's unusual combination of `fork()` and `exec()` for process creation was an inspired design. In this paper, we argue that fork was a clever hack for machines and programs of the 1970s that has long outlived its usefulness and is now a liability. We catalog the ways in which fork is a terrible abstraction for the modern programmer to use, describe how it compromises OS implementations, and propose alternatives.

As the designers and implementers of operating systems, we should acknowledge that fork's continued existence as a first-class OS primitive holds back systems research, and deprecate it. As educators, we should teach fork as a historical artifact, and not the first process creation mechanism students encounter.

ACM Reference Format:
Andrew Baumann, Jonathan Appavoo, Orran Krieger, and Timothy Roscoe. 2019. A `fork()` in the road. In *Workshop on Hot Topics in Operating Systems (HotOS '19), May 13–15, 2019, Bertinoro, Italy.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3317550.3321435

50 years later, fork remains the default process creation API on POSIX: Atlidakis et al. [8] found 1304 Ubuntu packages (7.2% of the total) calling fork, compared to only 41 uses of the more modern `posix_spawn()`. Fork is used by almost every Unix shell, major web and database servers (e.g., Apache, PostgreSQL, and Oracle), Google Chrome, the Redis key-value store, and even Node.js. The received wisdom appears to hold that fork is a good design. Every OS textbook we reviewed [4, 7, 9, 35, 75, 78] covered fork in uncritical or positive terms, often noting its "simplicity" compared to alternatives. Students today are taught that "the `fork` system call is one of Unix's great ideas" [46] and "there are lots of ways to design APIs for process creation; however, the combination of `fork()` and `exec()` are simple and immensely powerful … the Unix designers simply got it right" [7].

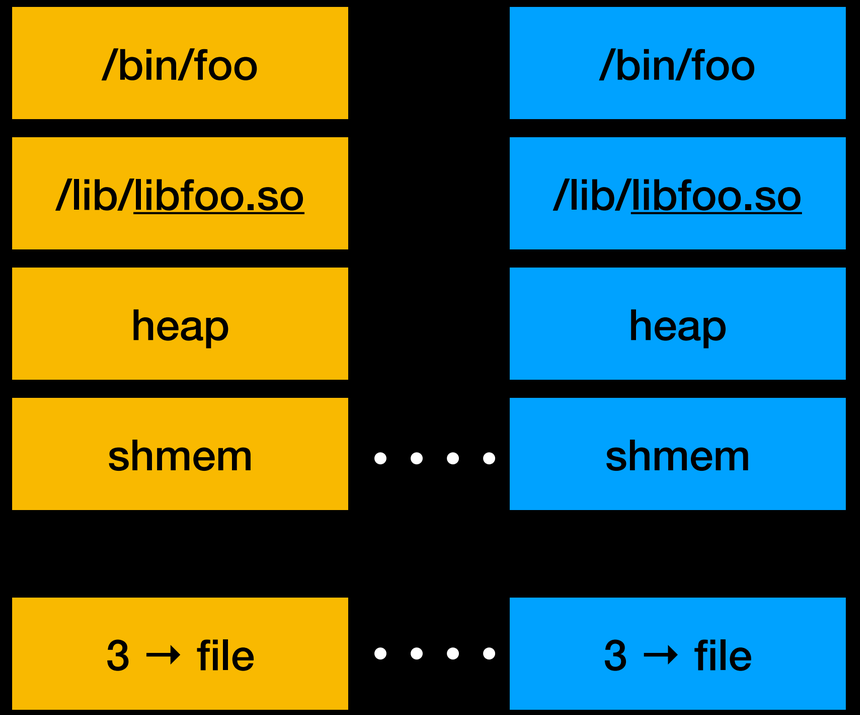
Our goal is to set the record straight. Fork is an anachronism: a relic from another era that is out of place in modern systems where it has a pernicious and detrimental impact. As a community, our familiarity with fork can blind us to its faults (§4). Generally acknowledged problems with fork include that it is not thread-safe, it is inefficient and unscalable, and it introduces security concerns. Beyond these limitations, fork has lost its classic simplicity; it today impacts all the

- fork() concept existed already in other systems, but early Unix simplified to the extreme
- Simple interface led to complex interactions with other features, slowness, new variants like vfork(), rfork(), clone()
- New systems should offer #2 and #3 only: a posix_spawn()-style interface for subprograms and otherwise threads

fork()

Create a new process like the parent

- Returns child PID in parent process, 0 in child process
- Copies* MAP_PRIVATE mappings (code, variables, stack, heap…)
- Shares MAP_SHARED mappings
- Duplicates file descriptors
- Other kernel resources and properties are … complicated



*see next…

Copy-on-write

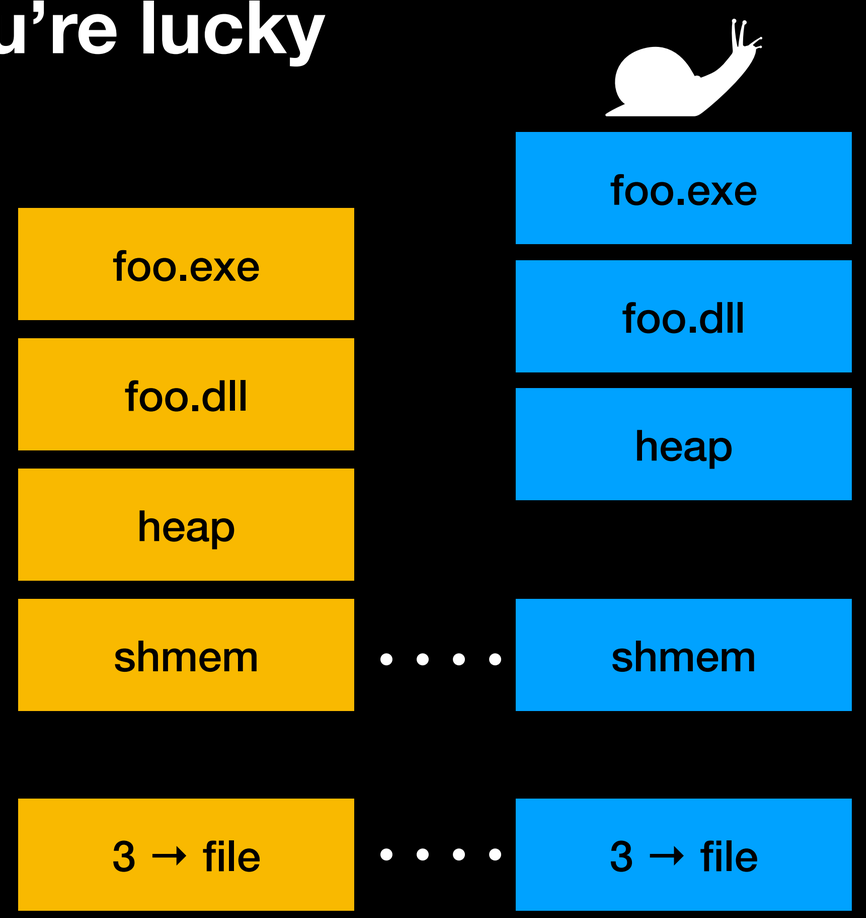
- In PDP Unix, *all* process memory was literally copied
- Since the VAX era: copy-on-write (see also: overcommit)
- The page table is still often copied on fork() and occupies memory
  - Linux huge pages share page tables*, but work is ongoing for default pages
  - Number of pages involved depends on configuration, libraries, etc and can be very large!

*perhaps not as well as it could?

Simulating fork() on Windows

Enough to make PostgreSQL work if you're lucky

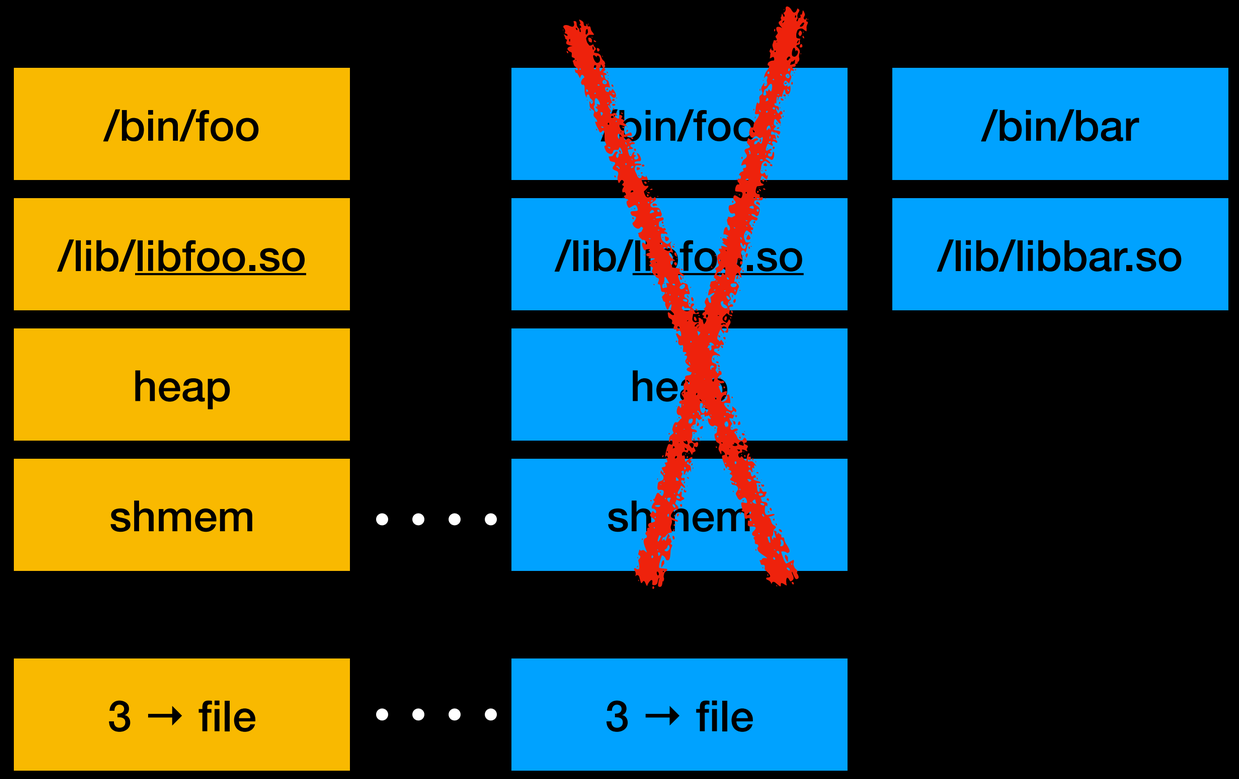
- The only way to create a child process is CreateProcess(), which shares selected handles but not memory with the child
- The contents of the important global variables have to be restored by hand, libraries initialize themselves from scratch, etc…
- The addresses of private mappings may be different, but we don't care about those
- The main shared memory region must be at the same address in all backends, so we jump through a number of hoops, even retrying
- Reported to add ~40ms to parallel queries with tiny memory map, probably much more depending on configured libraries and page count



fork() + exec()

Create a new process to run a different program

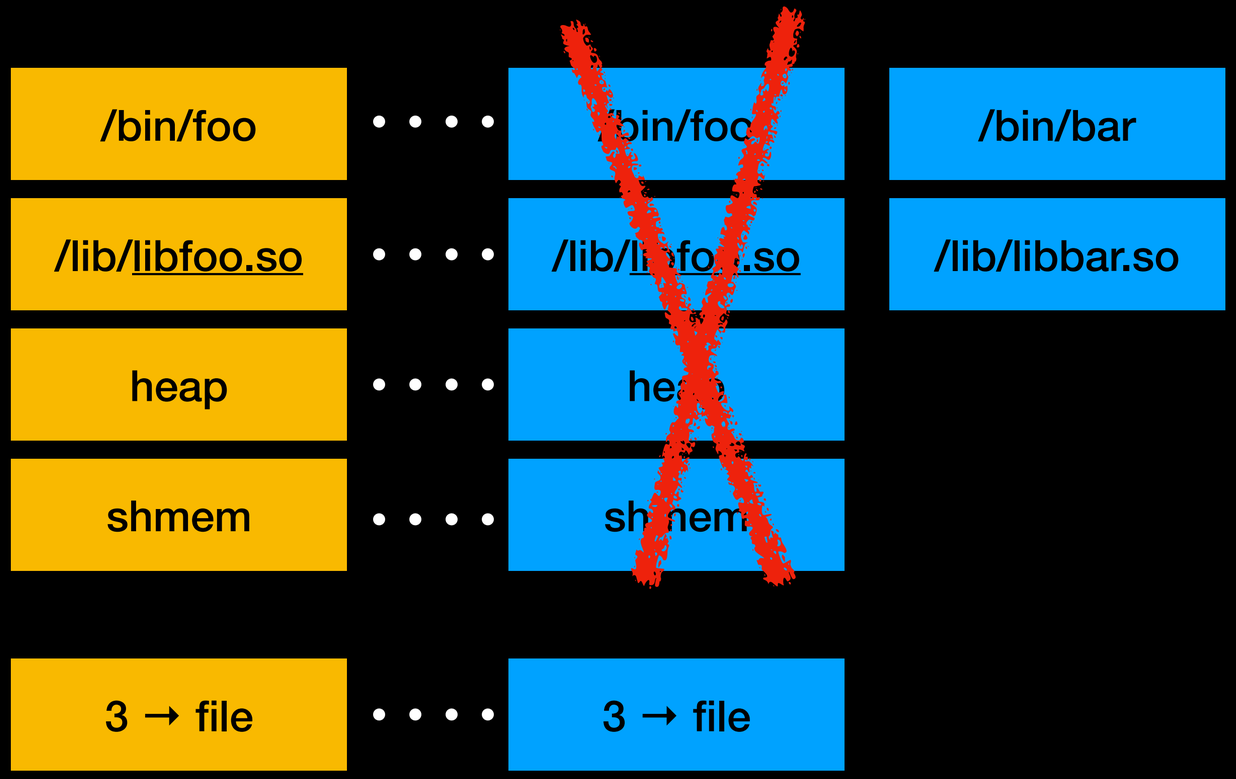
- In pre-VAX Unixen, very slow due to wasted temporary copy
- Even today, if page tables can't be shared, must perform copying in proportion to the number of pages, just to throw the copy away



vfork() + exec()

Create a new process to run a different program… faster

- Borrow the parent's memory map to skip useless overheads
- Child is only allowed to call exec() or exit(), and control doesn't return to the parent until then (among other reasons, it shares the same stack so concurrency can't work)
- POSIX removed vfork() and supplied posix_spawn(), but it lives on as an implementation detail called by many libc system(), popen(), posix_spawn()



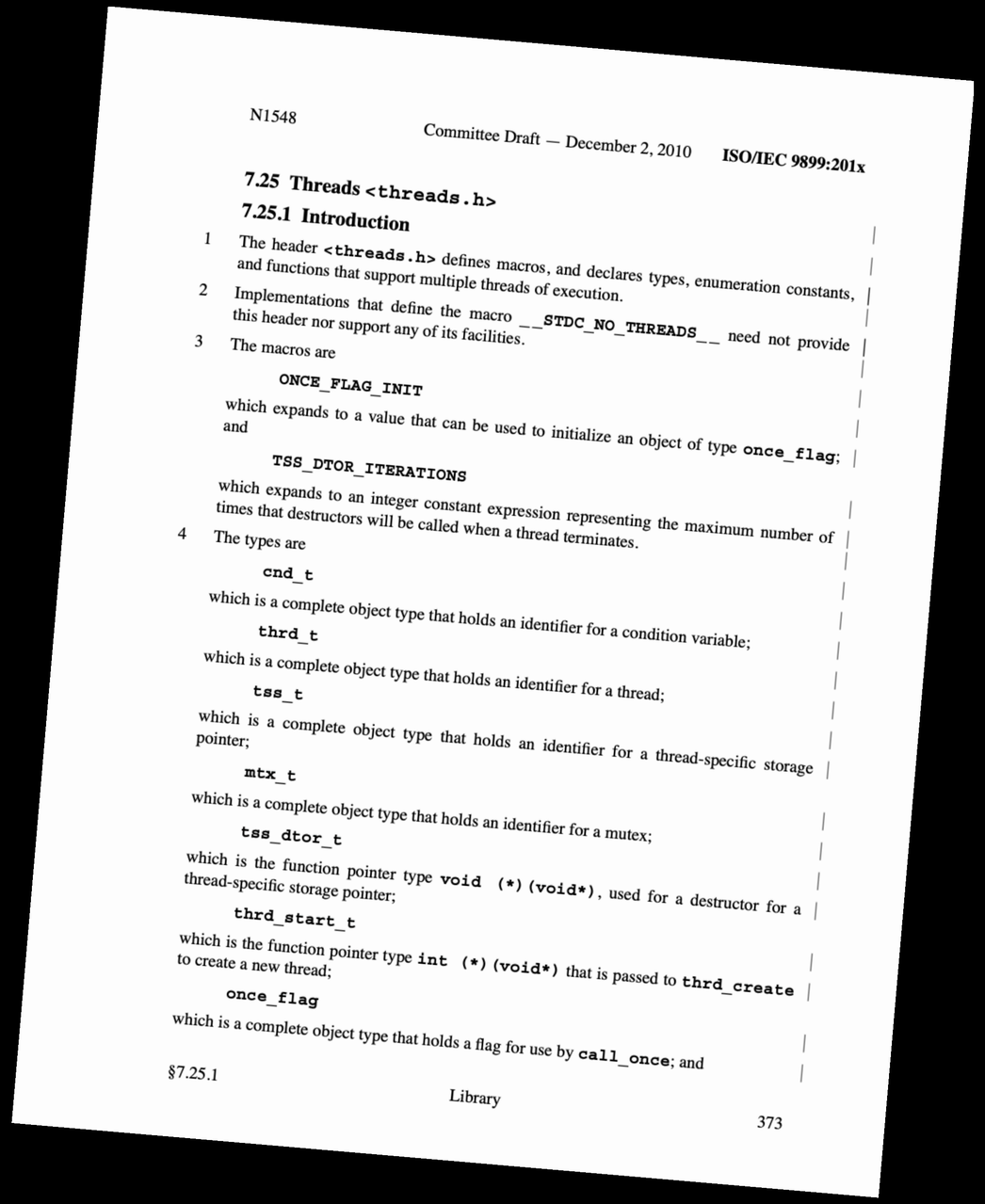
pthread_create()

Create a new thread of execution in the current process

- Identified by opaque pthread_t handles
- Global variables, file descriptors accessible (with significant new problems, see later)
- Signals delivered to a process are handled by any thread not blocking, but can also be sent to a pthread_t within the process
- Context switching between those threads may be more efficient

C11 threads

- Missing on macOS, but clues spotted that it is coming
- MSVC 2022, but missing in MinGW (?)
- pthread_barrier_t equivalent missing (but easily implemented)
- Some static initialisers missing, despite being implementable on POSIX and Windows



7.25 Threads <threads.h>

7.25.1 Introduction

1 The header **<threads.h>** defines macros, and declares types, enumeration constants, and functions that support multiple threads of execution.

2 Implementations that define the macro **__STDC_NO_THREADS__** need not provide this header nor support any of its facilities.

3 The macros are

ONCE_FLAG_INIT

which expands to a value that can be used to initialize an object of type **once_flag**; and

TSS_DTOR_ITERATIONS

which expands to an integer constant expression representing the maximum number of times that destructors will be called when a thread terminates.

4 The types are

cnd_t

which is a complete object type that holds an identifier for a condition variable;

thrd_t

which is a complete object type that holds an identifier for a thread;

tss_t

which is a complete object type that holds an identifier for a thread-specific storage pointer;

mtx_t

which is a complete object type that holds an identifier for a mutex;

tss_dtor_t

which is the function pointer type **void (*)(void*)**, used for a destructor for a thread-specific storage pointer;

thrd_start_t

which is the function pointer type **int (*)(void*)** that is passed to **thrd_create** to create a new thread;

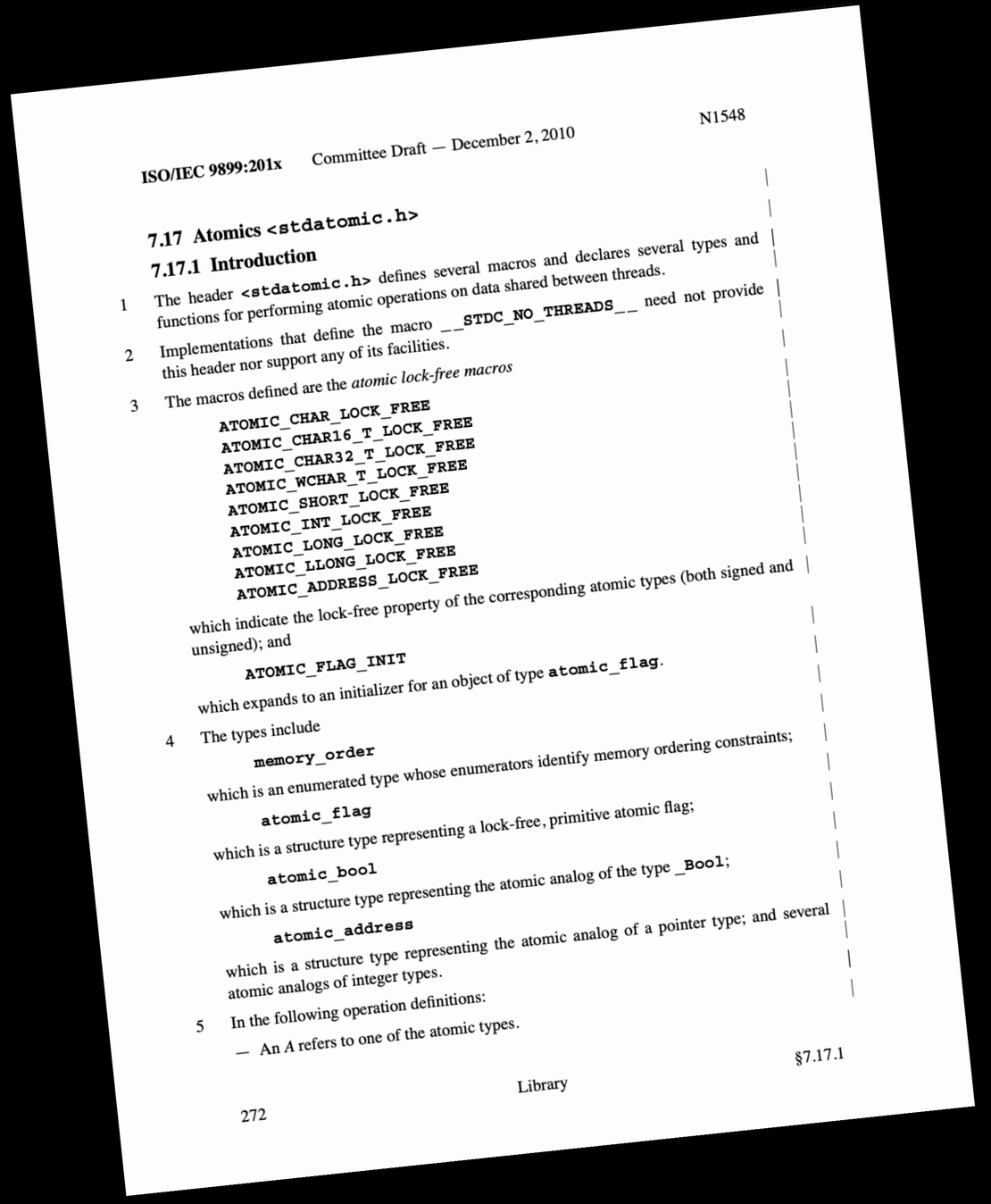
once_flag

which is a complete object type that holds a flag for use by **call_once**; and

C11 atomics

(Orthogonal really, just nearby)

- Widely supported, but needs MSVC 2022 on Windows
- All of port/atomics.h except spin lock delays can be redirected to <stdatomics.h>, deleting a lot of code
- Determining performance, and correctness implications of generated code, on many systems would take some effort, once we're ready to consider C11



ISO/IEC 9899:201x Committee Draft — December 2, 2010 N1548

7.17 Atomics <stdatomic.h>

7.17.1 Introduction

1 The header **<stdatomic.h>** defines several macros and declares several types and functions for performing atomic operations on data shared between threads.

2 Implementations that define the macro **__STDC_NO_THREADS__** need not provide this header nor support any of its facilities.

3 The macros defined are the *atomic lock-free macros*

ATOMIC_CHAR_LOCK_FREE
ATOMIC_CHAR16_T_LOCK_FREE
ATOMIC_CHAR32_T_LOCK_FREE
ATOMIC_WCHAR_T_LOCK_FREE
ATOMIC_SHORT_LOCK_FREE
ATOMIC_INT_LOCK_FREE
ATOMIC_LONG_LOCK_FREE
ATOMIC_LLONG_LOCK_FREE
ATOMIC_ADDRESS_LOCK_FREE

which indicate the lock-free property of the corresponding atomic types (both signed and unsigned); and

ATOMIC_FLAG_INIT

which expands to an initializer for an object of type **atomic_flag**.

4 The types include

memory_order

which is an enumerated type whose enumerators identify memory ordering constraints;

atomic_flag

which is a structure type representing a lock-free, primitive atomic flag;

atomic_bool

which is a structure type representing the atomic analog of the type **_Bool**;

atomic_address

which is a structure type representing the atomic analog of a pointer type; and several atomic analogs of integer types.

5 In the following operation definitions:

— An *A* refers to one of the atomic types.

§7.17.1 Library

272

Some projects that made me miss threads a lot…

Processes vs dynamic shared memory

dsa.c must die

My EDB blog from 2018

- PostgreSQL is based on separate processes and shared memory. Since the project's origins, there has been a shared memory area mapped into every process at the same address, but it is of fixed size. To support Parallel Query, Robert Haas, Amit Kapila and Noah Misch had already developed Dynamic Shared Memory (DSM) as a way to create more shared memory segments any time. Unfortunately those segments are mapped at a different address in every process that attaches to them, and there was no facility for allocating and freeing small objects within them. I took a prototype memory allocator called sb_alloc written by Robert Haas and used it to create a new facility called Dynamic Shared Areas (DSA), which provides a malloc/free interface backed by an expandable set of DSM segments. The weirdest thing about DSA is that it works in terms of dsa_pointer, and these pointers must be converted to a backend-local address before they can be dereferenced. Maybe one day PostgreSQL will use threads, or some other trick to get dynamic shared memory mapped at the same address in all processes, at which point this extra address decoding hop can be removed. Until then, something like this seemed to be necessary to support the sharing of variable sized data between backends, as required for Parallel Hash. (DSA is now also used to support several other features including Parallel Bitmap Heap Scan.)

Processes vs asynchronous I/O

Cross-process I/O completion hurts

- Backends need to be able to consume completions for I/Os started by other backends. io_uring can do that if you share user space queue and descriptor.
- Designers of other relevant APIs didn’t conceive of such madness:
  - I/O Completion Ports (Windows)
  - IoRing (Windows)
  - POSIX AIO (FreeBSD and maybe more)
- You can make them work with enough engineering and some performance loss (prototypes exist), but…

Other things you can’t do with processes

- Windows’ futexes don’t work cross-process
- macOS and some others don’t support unnamed semaphores cross-process (pshared=1 is optional in POSIX)
- Ditto for POSIX mutexes, condition variables, barriers (PTHREAD_PROCESS_SHARED is optional in POSIX)

Architectures

Architecture of a Database System

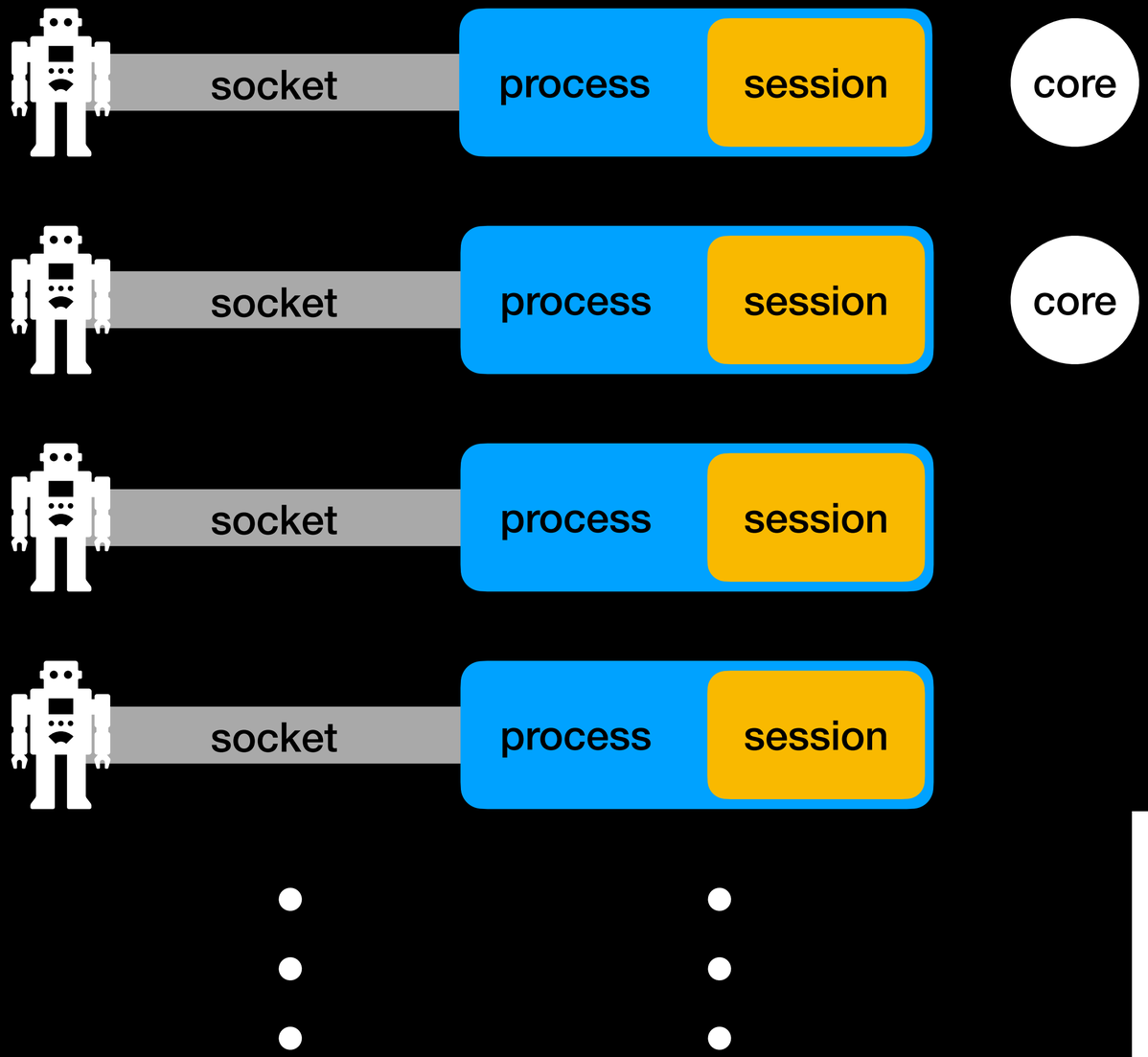
Joseph M. Hellerstein[1], Michael Stonebraker[2] and James Hamilton[3]

[1] *University of California, Berkeley, USA, hellerstein@cs.berkeley.edu*
[2] *Massachusetts Institute of Technology, USA*
[3] *Microsoft Research, USA*

Abstract

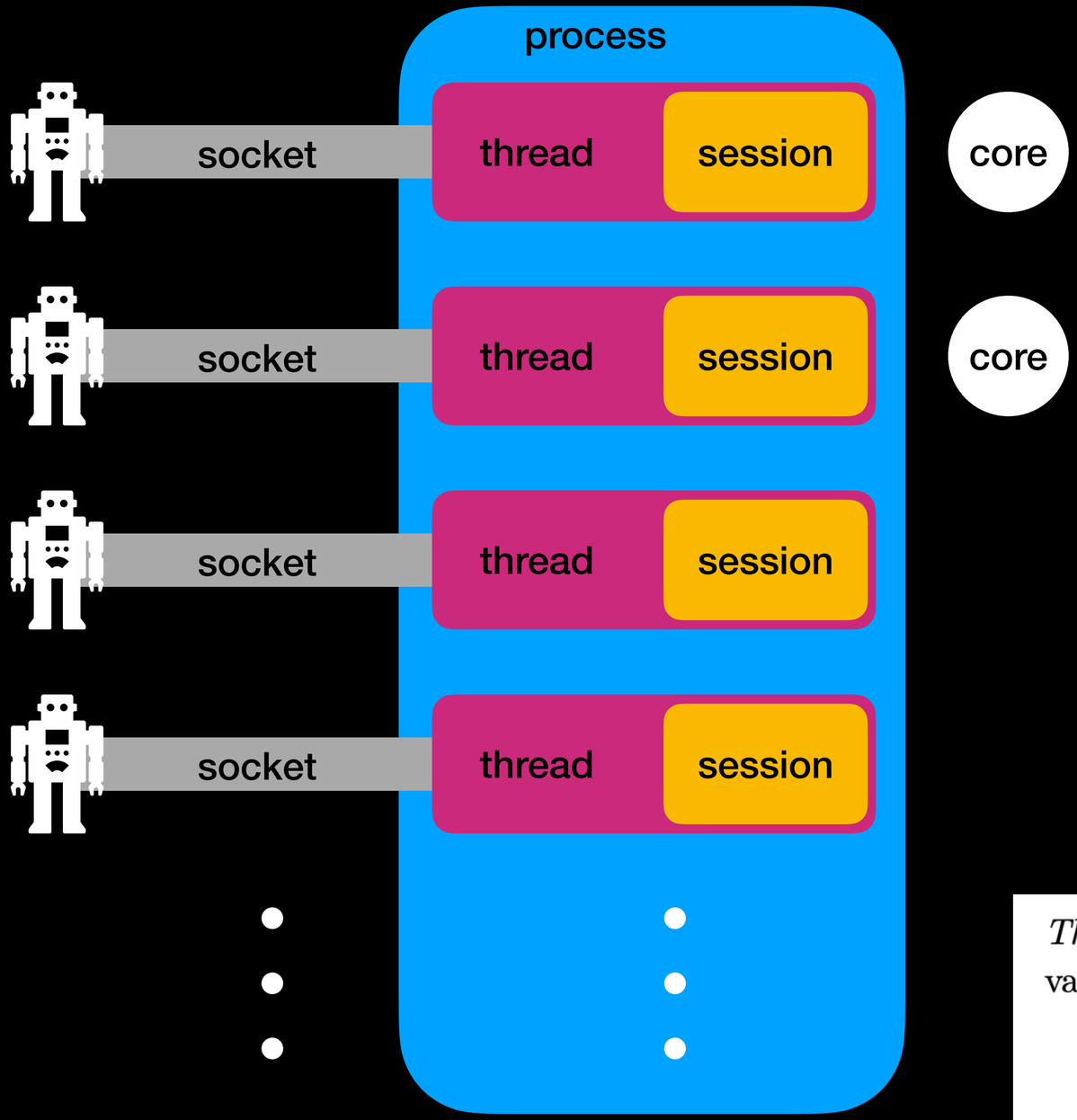
Database Management Systems (DBMSs) are a ubiquitous and critical component of modern computing, and the result of decades of research and development in both academia and industry. Historically, DBMSs were among the earliest multi-user server systems to be developed, and thus pioneered many systems design techniques for scalability and reliability now in use in many other contexts. While many of the algorithms and abstractions used by a DBMS are textbook material, there has been relatively sparse coverage in the literature of the systems design issues that make a DBMS work. This paper presents an architectural discussion of DBMS design principles, including process models, parallel architecture, storage system design, transaction system implementation, query processor and optimizer architectures, and typical shared components and utilities. Successful commercial and open-source systems are used as points of reference, particularly when multiple alternative designs have been adopted by different groups.

- 2007 overview of database architectural choices
- Some of the following citations are thus surely obsolete by now… except the ones about us



Process per DBMS worker:
This is the most straight-forward process model and is still heavily used today. DB2 defaults to *process per DBMS worker* on OSs that do not support high quality, scalable OS threads and *thread per DBMS worker* on those that do. This is also the default Oracle process model. Oracle also supports *process pool* as described below as an optional model. PostgreSQL runs the *process per DBMS worker* model exclusively on all supported operating systems.

Realistic thread model



Thread per DBMS worker: This is an efficient model with two major variants in use today:

1. *OS thread per DBMS worker*: IBM DB2 defaults to this model when running on systems with good OS thread support and this is the model used by MySQL.

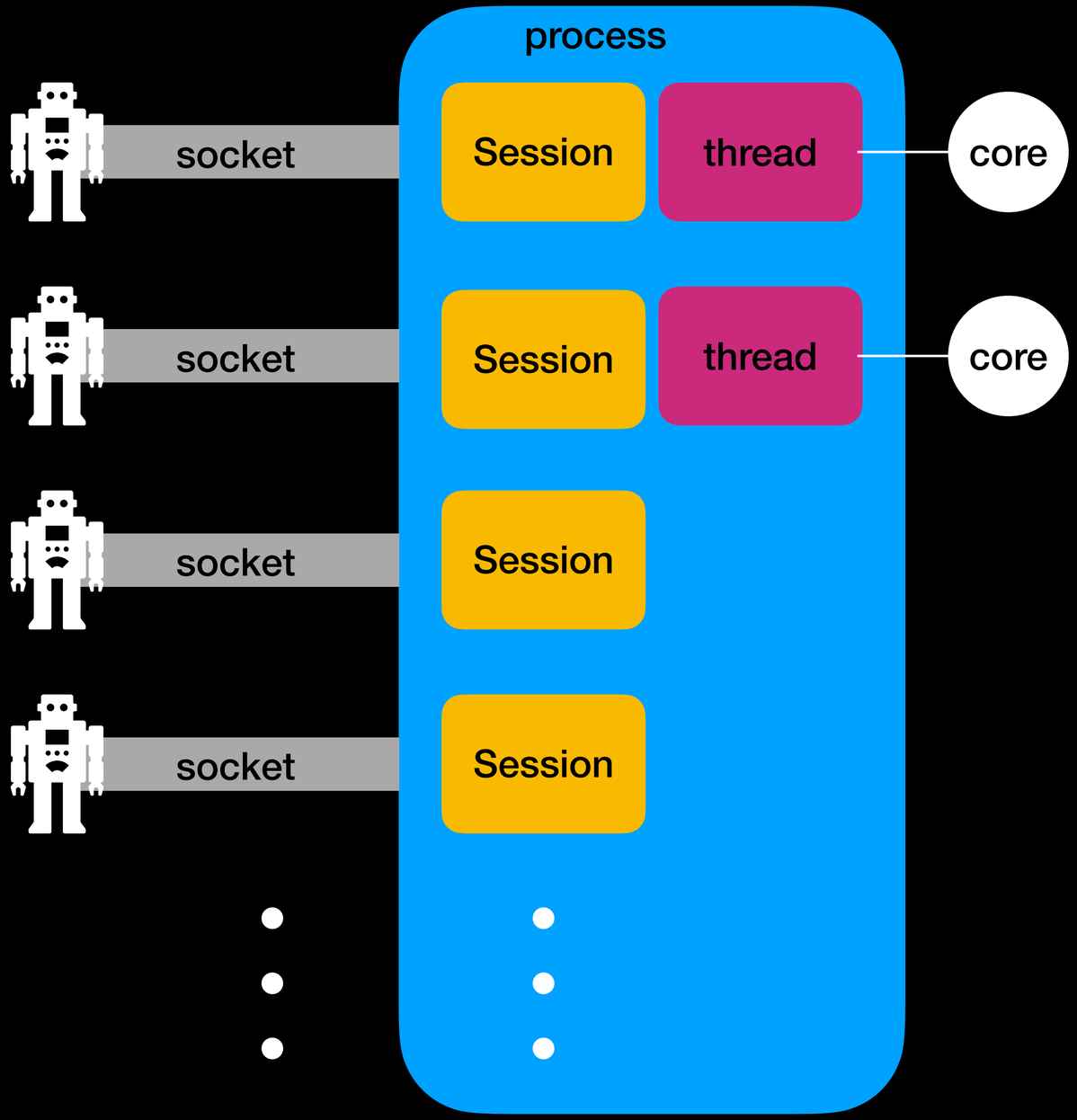
2. *DBMS thread per DBMS worker*: In this model, DBMS workers are scheduled by a lightweight thread scheduler on either OS processes or OS threads. This model avoids any potential OS scheduler scaling or performance problems at the expense of high implementation costs, poor development tools support, and substantial long-standing software maintenance costs for the DBMS vendor. There are two main sub-categories of this model:
    a. *DBMS threads scheduled on OS process*: A lightweight thread scheduler is hosted by one or more OS processes. Sybase supports this model as does Informix. All current generation systems using this model implement a DBMS thread scheduler that schedules DBMS workers over multiple OS processes to exploit multiple processors. However, not all DBMSs using this model have implemented *thread migration*: the ability to reassign an existing DBMS thread to a different OS process (e.g., for load balancing).
    b. *DBMS threads scheduled on OS threads*: Microsoft SQL Server supports this model as a non-default option (default is *DBMS workers multiplexed over a thread pool* described below). This SQL Server option, called *Fibers*, is used in some high scale transaction processing benchmarks but, otherwise, is in fairly light use.

Bad idea

```
Green thread 1:
main()
 foo()
  foox()

Green thread 2:
main()
 bar()
  barx()
```

- Mostly obsolete* concept from uniprocessor days, generalized to so called M:N thread model on top of kernel thread
- Context switching was managed by libc or application with obsolete getcontext() etc functions or horrible non-portable code
- Windows has “fibers”

*as far as C is concerned



Holy grail?

Process/thread pool:
In this model, DBMS workers are multiplexed over a pool of processes. As OS thread support has improved, a second variant of this model has emerged based upon a *thread pool* rather than a *process pool*. In this latter model, DBMS workers are multiplexed over a pool of OS threads:

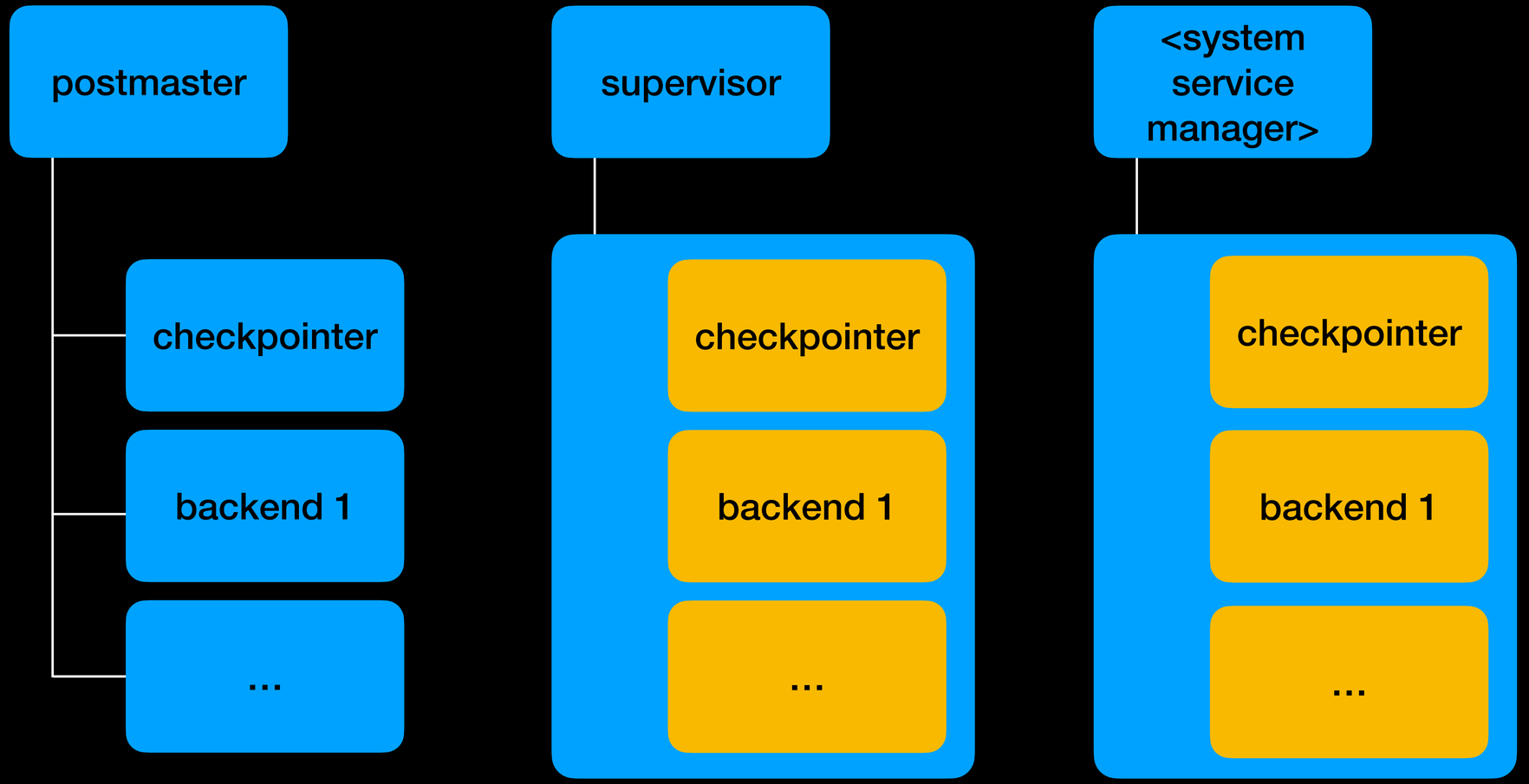
1. *DBMS workers multiplexed over a process pool*: This model is much more memory efficient than *process per DBMS worker*, is easy to port to OSs without good OS thread support, and scales very well to large numbers of users. This is the optional model supported by Oracle and the one they recommend for systems with large numbers of concurrently connected users. The Oracle default model is *process per DBMS worker*. Both of the options supported by Oracle are easy to support on the vast number of different OSs they target (at one point Oracle supported over 80 target OSs).
2. *DBMS workers multiplexed over a thread pool*: Microsoft SQL Server defaults to this model and over 99% of the SQL Server installations run this way. To efficiently support tens of thousands of concurrently connected users, as mentioned above, SQL Server optionally supports *DBMS threads scheduled on OS threads*.

```
-int                    io_method = DEFAULT_IO_METHOD;
-int                    io_max_concurrency = -1;
+postmaster_guc int                     io_method = DEFAULT_IO_METHOD;
+postmaster_guc int                     io_max_concurrency = -1;


-PgAioCtl   *pgaio_ctl;
+pg_global PgAioCtl   *pgaio_ctl;


-PgAioBackend *pgaio_my_backend;
+session_local PgAioBackend *pgaio_my_backend;
```

```
/*
 * A struct encapsulating some elements of a user's session.  For now this
 * manages state that applies to parallel query, but in principle it could
 * include other things that are currently global variables.
 */
typedef struct Session
{
    dsm_segment *segment;       /* The session-scoped DSM segment. */
    dsa_area   *area;           /* The session-scoped DSA area. */

    /* State managed by typcache.c. */
    struct SharedRecordTypmodRegistry *shared_typmod_registry;
    dshash_table *shared_record_table;
    dshash_table *shared_typmod_table;
} Session;
```

Attempts using global → thread_local

- Heikki's current branch: https://github.com/hlinnaka/postgres/tree/threading
- Postgres Pro prototype
- CMU Peloton (also ported to C++, another thing Berkeley POSTGRES deferred)
- Multiple projects to port to Windows via that route
- A developer who gave up two years ago after reading "Features we don't want" on our Wiki!
- A report of a commercial product in Japan (anyone know what that is?)
- I myself prototyped hack-grade parallel query with threads
- Probably many more!


postmaster
checkpointer
backend 1
...
supervisor
checkpointer
backend 1
...
<system service manager>
checkpointer
backend 1
...

WIP

- Heikki’s branch: https://github.com/hlinnaka/postgres
- Smaller pieces being developed independently

General goals

- Support both process and thread mode for some time, so that extensions have time to adjust
- Be fully portable
- Stabilise, study

Non-goals for prototype

In other words, be as much like process model as possible

- Sharing file descriptors
- Sharing relcache, syscache
- Sharing MemoryContexts between backends
- Removing DSM, DSA
- Removing all the serialization of state for parallel query workers
- Removing the fake signal system from Windows backends

pg_threads.h

Minimal harmonisation of POSIX and Windows APIs

- Even if we adopted other parts of C11, it seems a bit too soon to use <threads.h>
- Use C11 as naming guide, but add pg_ prefixes
- Require a way to implement pg_thread_local
- Add strangely missing static initializer macros (eg PG_MTX_STATIC_INIT)
- Patch previously proposed (CF #5194), will repost improved update soon
- Works out net zero-ish in line count because it obsoletes thread portability wrappers in pgbench, ecpg, libpq; more such opportunities exist

Removing non-mt safe code

- Many patches committed, eg:
  - Parser thread-safety
  - Removing dependencies on the global locale, preferring _l() functions, various workaround
  - Using _r() functions
  - Removing static buffers
  - Work continues!

Signalectomy

- SendInterrupt() proposal that will remove ProcSignal, CF #5118
- Experimental work tries other wakeup mechanisms for interrupts (pipes, futexes, custom io_uring, kqueue, iocp user events)
- Many cases of ad hoc signals are documented at:

  https://wiki.postgresql.org/wiki/Signals

PIDs in user-facing interfaces

A few places expose PIDs to users:

- pg_terminate_backend(<pid>)
- pg_stat_activity
- query cancellation

Plan:

- Replace PID with a thread id.
- Could use OS thread ID, but behavior isn't totally consistent across platforms
- Consensus at 2024.pgconf.dev seemed to be that inventing our own 32-bit thread ID would be better
- Possibly pgprocno + counter
- Need to make sure we don't reuse thread IDs too quickly, else we might terminate the wrong backend
- Potential standalone value: decoupling the relationship between thread and session could enable othe
- Robert volunteered to look into this at 2024.pgconf.dev

Global variables

Global variables in PostgreSQL fall into a few different categories:

- per-session state
- pointers to shared memory areas
- other variables that are initialized at postmaster startup and never change after that
- constants

Many global variables are used to hold GUCs, and they can fall into any of those categories dependi
PGC_POSTMASTER or PGC_USERSET.

The plan:

- Add annotations to all global variables, to mark which category they fall into. Variables holding pe
  thread-local variables. The distinction between the other categories is just for documentation pur
- Provide a tool that can list all global variables that are missing the annotations (pgguclifetimes). A
  ideally. This will also be useful for extension authors to find missing annotations in extensions.
  - alternative idea: could we use a simple perl script for this, like we do for the PGDLLIMPORT
- Potential standalone value:
  - "modified once" annotated variables could be put in a separate binary section which would improve performance
  - Could help with the engineering overhead of figuring out where to initialize at postmaster startup

Survey of signal uage

Sender→receiver	Signal	Description	Replacement	Patch
administrator→postmaster	SIGTERM	smart shutdown	keep, and add control socket?	
administrator→postmaster	SIGQUIT	immediate shutdown	keep, and add control socket?	
administrator→postmaster	SIGINT	fast shutdown	keep, and add control socket?	
administrator→postmaster	SIGHUP	reload	keep, and add control socket?	
postmaster→all	SIGQUIT	immediate shutdown, quickdie(), _exit()	keep	
postmaster→all	SIGTERM	exit at next CFI()	SendInterrupt(INTERRUPT_DIE)?	
postmaster→avlauncher	SIGUSR2	start autovacuum	?	
postmaster→checkpointer	SIGUSR2	shutdown	?	
postmaster→walsenders	SIGUSR2	finish and shutdown	?	
postmaster→pgarch	SIGUSR2	shutdown	?	
postmaster→startup	SIGUSR2	promote	?	
postmaster→client backend	SIGINT	cancel query	SendInterrupt(INTERRUPT_CANCEL)?	
postmaster→all	SIGKILL	timed out while waiting for shutdown	keep	
postmaster→any	SIGUSR1	bgworker state change notification	SetLatch() (does this need to be 'robust'?)	SendInterrupt() proposal
postmaster→all	SIGHUP	reload config	SendInterrupt(INTERRUPT_RELOAD_CONFIG)?	
kernel→postmaster	SIGCHLD	child state change notification	keep, but refactor?	
kernel→backend	SIGINFO	postmaster exited	keep for now (but see below for MT redesign ideas)	
kernel→backend	SIGALRM	itimer	keep for now, but change handlers to do RaiseInterrupt(INTERRUPT_XXX)	SendInterrupt() proposal
backend→backend	SIGURG	latch wakeup	keep for now, but later replace with ?	
backend→backend	SIGUSR1	SendProcSignal(pid, PROCSIG_XXX)	SendInterrupt(INTERRUPT_XXX, procno)	SendInterrupt() proposal
backend→postmaster	SIGUSR1	SendPostmasterSignal(PMSIGNAL_XXX)	?	

Main themes: co-operative interrupts, latches as universal wakeup, robust supervisor

As discussed in the Vancouver Multithreading meeting, one of our subtasks will be to remove the dependency on process IDs, kill() and signal handlers.

- PostgreSQL used to use signal handlers to *do* things, but now it only uses them to set flags, to remember to do things later at the next CHECK_FOR_INTERRUPTS()
- The remaining cases of doing something more in a handler are:
  - fast _exit(), but in a multithreaded future that would move out to container-level (backends won't have to exit one at a time)
  - timer-related activities, but in a multithreaded future those would presumably move out to a new timer infrastructure
- PostgreSQL used to use signals to interrupt blocking slow system calls, but that was inherently racy, and now everything should be nonblocking
- Latches currently use signals as an implementation detail on Unix (but not always an actual handler), but that could change if required
- The postmaster tries to avoid relying on shared memory contents and is not allowed to use the latch infrastructure, because we want the postmaster to be able to coordinate a crash-restart even if memory is corrupted

fin